

Project - Build a Student Grade & Honor Roll Tracker.....	1
Task 1: Read messy Student Names from user	2
Task 2: Request user to input Math and Science grade and calculate the average	3
Task 3: Scale Up to a Full Schedule.....	4
Task 4: Clean the Main Architecture (Introducing Multiple Functions).....	5
Task 5: OOPs concepts – class, encapsulation.....	7
Task 6: OOPs concepts – Data hiding.....	9
Task 7: OOPs – Inheritance and overriding	12
Task 8: OOPS – Polymorphism	15
Task 9: File Input Output	18
Task 10: Read back from on start-up.	21

Project - Build a Student Grade & Honor Roll Tracker

This project bridges your fundamental knowledge (I/O, logic, and functions) with the next set of essential Java concepts: **Strings, Arrays, Wrapper Classes/Type Casting**, and **Basic Classes/Objects**, inheritance and Polymorphism.

The Goal

Build an application that reads a student's raw data string from user.

Extracts their numeric grades into an array, calculates their average, and generates an automated report profile card.

This UnifiedTracker.java step-by-step evolution of GradeTracker application.

Task 1: Read messy Student Names from user

- Concept: Strings and String Manipulation (.trim(), .substring(), .toUpperCase())
- Goal: Capture a messy text string from the keyboard, clean it, and format it properly.

```
import java.util.Scanner;

public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // Clean and format text
        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);

        System.out.println("Saved Name: " + formattedName + "");

        // [Task 2 code will be inserted directly here next]
        inputScanner.close();
    }
}
```

Step B: Use the Object inside your Main Runner program

Update your main running file (GradeTracker.java) to make a live instance from your blueprint:

```
public class GradeTracker {
    public static void main(String[] args) {

        // 1. Prepare data variables
        String processedName = "Alex Smith";
        int[] scores = {95, 88, 91, 94};

        // 2. OOP Concept: Instantiate a real Student object container from our blueprint
        Student currentStudent = new Student(processedName, scores);

        // 3. Ask the object to execute its embedded logic method
        System.out.println("=== Final Report Card ===");
        System.out.println("Student Profile Name: " + currentStudent.name);
        if (currentStudent.qualifiesForHonorRoll()) {
            System.out.println("Status: CONGRATULATIONS! Placed on the Academic Honor Roll.");
        } else {
            System.out.println("Status: Satisfactory standing.");
        }
    }
}
```

Task 2: Request user to input Math and Science grade and calculate the average

New Concepts: Type Casting & Wrapper Classes (Integer.parseInt())

Parse the use given math integers, and use type casting for a precise decimal average.

```
import java.util.Scanner;

public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
        System.out.println("Saved Name: '" + formattedName + "'");

        // — ADDING TASK 2 NEW CODE DIRECTLY BELOW —
        System.out.println("\n=== Task 2: Parsing & Casting ===");
        System.out.print("Enter text grade for Math: ");
        String mathText = inputScanner.next();
        System.out.print("Enter text grade for Science: ");
        String scienceText = inputScanner.next();

        // Wrapper parsing
        int mathScore = Integer.parseInt(mathText);
        int scienceScore = Integer.parseInt(scienceText);

        // Type casting for decimal average
        double initialAverage = (double) (mathScore + scienceScore) / 2;
        System.out.println("Initial 2-Class Average: " + initialAverage + "%");

        // [Task 3 code will be inserted directly here next]
        inputScanner.close();
    }
}
```

Task 3: Scale Up to a Full Schedule.

- Concept: (Arrays & Loops)
- Goal: Instead of calculating just Math and Science as loose variables.
Ask how many total classes they have, create an **Array** of that size, and use a for loop to capture everything dynamically.

```
import java.util.Scanner;

public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Task 1: String Input & Cleaning ===");
        System.out.print("Enter student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        String cleanName = rawName.trim();
        String formattedName = cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
        System.out.println("Saved Name: '" + formattedName + "'");

        System.out.println("\n=== Task 2: Parsing & Casting ===");
        System.out.print("Enter text grade for Math: ");
        String mathText = inputScanner.next();
        System.out.print("Enter text grade for Science: ");
        String scienceText = inputScanner.next();

        int mathScore = Integer.parseInt(mathText);
        int scienceScore = Integer.parseInt(scienceText);

        double initialAverage = (double) (mathScore + scienceScore) / 2;
        System.out.println("Initial 2-Class Average: " + initialAverage + "%");

        // — ADDING TASK 3 NEW CODE DIRECTLY BELOW —
        System.out.println("\n=== Task 3: Arrays & Loops ===");
        System.out.print("How many total classes do you want to track in the array? ");
        int totalClasses = inputScanner.nextInt();

        int[] scheduleGrades = new int[totalClasses];
        int sum = 0;

        for (int i = 0; i < scheduleGrades.length; i++) {
            System.out.print("Enter text grade for class #" + (i + 1) + ": ");
            String textGrade = inputScanner.next();
            // We combine Task 2's Wrapper parsing directly inside our array loop!
            scheduleGrades[i] = Integer.parseInt(textGrade);
            sum += scheduleGrades[i];
        }

        double finalAverage = (double) sum / scheduleGrades.length;
        System.out.println("Final Array Schedule Average: " + finalAverage + "%");

        // [Task 4, 5, and 6 will now pull this data into an Object architecture below]
        inputScanner.close();
    }
}
```

Task 4: Clean the Main Architecture (Introducing Multiple Functions)

Introduce **4 helper functions** outside of `main`. We then update `main` to pass its data straight into these functions, keeping the execution flow clean and readable.

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

public class UnifiedTracker {

    // — TASK 4: FUNCTION 1 (Extracts Text Cleaning Logic from Task 1) —
    public static String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    // — TASK 4: FUNCTION 2 (Extracts Parsing & Casting Logic from Task 2) —
    public static double parseAndCalculateInitial(String textGrade1, String textGrade2) {
        int score1 = Integer.parseInt(textGrade1);
        int score2 = Integer.parseInt(textGrade2);
        return (double) (score1 + score2) / 2;
    }

    // — TASK 4: FUNCTION 3 (Extracts Array Loop Math Logic from Task 3) —
    public static double calculateAverage(int[] gradesArray) {
        int sum = 0;
        for (int score : gradesArray) {
            sum += score;
        }
        return (double) sum / gradesArray.length;
    }

    // — NEW TASK 4: FUNCTION 4 (Replaces and Handles the Task 3 For-Loop Input) —
    public static void fillGradesArray(Scanner inputScanner, int[] scheduleGrades) {
        for (int i = 0; i < scheduleGrades.length; i++) {
            System.out.print("Enter text grade for class #" + (i + 1) + ": ");
            String textGrade = inputScanner.next();
            // Performs the Task 2 Wrapper parsing directly into the array element
            scheduleGrades[i] = Integer.parseInt(textGrade);
        }
    }
}
```

```

// — CLEAN MAIN EXECUTIVE RUNNER —
public static void main(String[] args) {
    Scanner inputScanner = new Scanner(System.in);

    // --- TASK 1 CUMULATIVE REUSE ---
    System.out.println("=== Task 1: String Input & Cleaning ===");
    System.out.print("Enter student name (e.g., ' john doe '): ");
    String rawName = inputScanner.nextLine();

    String formattedName = formatName(rawName); // Function Call 1
    System.out.println("Saved Name: '" + formattedName + "'");

    // --- TASK 2 CUMULATIVE REUSE ---
    System.out.println("\n=== Task 2: Parsing & Casting ===");
    System.out.print("Enter text grade for Math: ");
    String mathText = inputScanner.next();
    System.out.print("Enter text grade for Science: ");
    String scienceText = inputScanner.next();

    double initialAverage = parseAndCalculateInitial(mathText, scienceText); // Function Call 2
    System.out.println("Initial 2-Class Average: " + initialAverage + "%");

    // --- TASK 3 CUMULATIVE REUSE (With Loop Replaced) ---
    System.out.println("\n=== Task 3 & 4: Array Size & Loop Function ===");
    System.out.print("How many total classes do you want to track? ");
    int totalClasses = inputScanner.nextInt();

    // Declare the blank fixed-size array container
    int[] scheduleGrades = new int[totalClasses];

    // Function Call 4: Loops and populates the scheduleGrades array from inside the function
    fillGradesArray(inputScanner, scheduleGrades);

    // --- TASK 4 OUTPUT VERIFICATION ---
    System.out.println("\n=== Task 4: Final Calculated Return Values ===");
    double computedAverage = calculateAverage(scheduleGrades); // Function Call 3
    System.out.println("Clean Function Calculated Average: " + computedAverage + "%");

    inputScanner.close();
}
}

```

Task 5: OOPs concepts – class, encapsulation

- Concept: OOP - Encapsulation
- Goal: Instead of loose data and method. Create a Class Student with

Data:	Student name, grades
Subject list	static String[] containing your subject names: Math, Science, Social, English, and French
Constructor	In Constructor, ask user to input grades for above subjects.
Private Method	fillGradesArray() Instead of asking the user for a raw total number of classes, the program automatically loops through these specific subjects, prompts for each one by name, and maps them to their respective positions in the array.
Private Method	Instead of formatting in Class Main. Format here. private String formatName(String rawName)
Public Method	calculateAverage() Loop through data: grades[] and calculate grade.

```
// — MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- DATA COLLECTION INTAKE ---
        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // --- OBJECT BIRTH SYNTHESIS ---
        // We instantiate the object. It handles text cleaning and sequential
        // subject tracking completely on its own inside the constructor!
        Student activeStudent = new Student(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n=====");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeStudent.name);
        System.out.println("Total Classes:   " + activeStudent.grades.length);

        // Call instance method to compute arithmetic output
        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:   " + computedAverage + "%");
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — FULLY ENCAPSULATED OBJECT BLUEPRINT —
class Student {
    // Shared structural template for subjects
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    // Attributes
    public String name;
    public int[] grades;

    // — REFACTORED AUTOMATED CONSTRUCTOR —
    public Student(String rawName, Scanner inputScanner) {
        // 1. Call private helper to format text upon birth
        this.name = formatName(rawName);

        // 2. Initialize internal array capacity to match the total number of subjects
        this.grades = new int[SUBJECTS.length];

        // 3. Call private helper to run subject-by-subject loops and error safeguards
        fillGradesArray(inputScanner);
    }

    // — PRIVATE INSTANCE HELPER METHODS —

    // Function 1: Pure internal text processing (Private, instance method)
    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    // Function 2: Enumerates through subject strings directly into 'this.grades'
    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            // Prompts specifically by the subject name
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();

            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("❌ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    // Function 3: Mathematical engine targeting 'this.grades' automatically
    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```

Task 6: OOPs concepts – Data hiding

Now that your fundamental structural framework is locked down, let me know if you would like to proceed with **The Data Hiding Step**: Converting public String name to private String name and creating an official **Getter Method** (public String getName()) to implement proper professional Java security standards.

- **Private Fields:** name and grades fields are changed to private. The main method can no longer access activeStudent.name or activeStudent.grades.length directly without causing a compilation error.
- **Getter Methods:** We introduce getName() and getGrades() to allow controlled, read-only access to our private fields. By using **Getters**, your main program gets the variables it needs, but the object maintains absolute authority over its internal state.
- **Zero Code Discarded:** The fixed SUBJECTS array layout, error-trapping inputs, and formatting mechanics remain fully intact

```
// — MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // Instantiating the object profile
        Student activeStudent = new Student(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n=====");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("=====");

        // — UPGRADED ACCESS VIA GETTER METHOD CALLS —
        // Writing activeStudent.name here would now cause a Java compilation error!
        System.out.println("Student Profile: " + activeStudent.getName());
        System.out.println("Total Classes:   " + activeStudent.getGrades().length);

        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:   " + computedAverage + "%");
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```

// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — FULLY ENCAPSULATED & HIDDEN OBJECT BLUEPRINT —
class Student {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    // — CONCEPT UPGRADE: CHANGED FROM PUBLIC TO PRIVATE (DATA HIDING) —
    private String name;
    private int[] grades;

    // Automated Constructor (Unchanged)
    public Student(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    // — NEW PUBLIC GETTER METHODS (THE DOORWAYS TO READ HIDDEN DATA) —

    // Allows outside classes to read the hidden name safely
    public String getName() {
        return this.name;
    }

    // Allows outside classes to read the hidden grades list size or values safely
    public int[] getGrades() {
        return this.grades;
    }

    // — PRIVATE INSTANCE HELPER METHODS (Fully Retained) —

    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }
}

```

```

private void fillGradesArray(Scanner inputScanner) {
    System.out.println("\n--- Entering Grades for Class Schedule ---");
    for (int i = 0; i < this.grades.length; i++) {
        System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
        String textGrade = inputScanner.next();

        try {
            this.grades[i] = Integer.parseInt(textGrade);
        } catch (NumberFormatException e) {
            System.out.println("✗ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
            this.grades[i] = 0;
        }
    }
}

public double calculateAverage() {
    int sum = 0;
    for (int score : this.grades) {
        sum += score;
    }
    return (double) sum / this.grades.length;
}

```

Task 7: OOPs – Inheritance and overriding

- Create a child class called **GraduateStudent** that extends your base Student. This allows the child class to automatically reuse the fields, getters, array logic, and validation rules from the parent class without rewriting a single line of that code.
- Add Unique Child Traits: We add a new private attribute unique to graduate students: private String thesisTitle, along with its own public getter (getThesisTitle()).
- introduce an status metric called **qualifiesForHonorRoll()** inside the parent class, which checks if a student has an average of **90.0% or higher**. Then, using **Method Overriding**, the GraduateStudent child class will rewrite this exact method to enforce a stricter rule: graduate students must maintain a **95.0% or higher** to make the honor roll.

```
// — MAIN RUNNER EXECUTIVE SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Graduate Student Data Entry ===");
        System.out.print("Enter raw student name (e.g., ' mariah carey '): ");
        String rawName = inputScanner.nextLine();

        System.out.print("Enter Research Thesis Topic: ");
        String thesis = inputScanner.nextLine();

        // Child Object Birth
        GraduateStudent activeGrad = new GraduateStudent(rawName, thesis, inputScanner);

        // --- REPORT GENERATION ---
        System.out.println("\n=====");
        System.out.println("      GRADUATE ACADEMIC REPORT      ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeGrad.getName());
        System.out.println("Total Classes:   " + activeGrad.getGrades().length);
        System.out.println("Research Focus:  " + activeGrad.getThesisTitle());

        double avg = activeGrad.calculateAverage();
        System.out.println("Final Average:   " + avg + "%");

        // — EVALUATING THE OVERRIDDEN METHOD —
        // Java executes the customized child method logic rules automatically!
        if (activeGrad.qualifiesForHonorRoll()) {
            System.out.println("Honor Roll:      ★ APPROVED (95%+ Target Cleared) ★");
        } else {
            System.out.println("Honor Roll:      Standard Graduate Standing");
        }
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```
// — INHERITANCE (CHILD CLASS) —
class GraduateStudent extends Student {

    private String thesisTitle;

    // Child Constructor calling parent super
    public GraduateStudent(String rawName, String thesisTitle, Scanner inputScanner) {
        super(rawName, inputScanner);
        this.thesisTitle = thesisTitle;
    }

    public String getThesisTitle() {
        return this.thesisTitle;
    }

    // — NEW STEP: METHOD OVERRIDING —
    @Override
    public boolean qualifiesForHonorRoll() {
        // Graduate students must hit a tougher threshold (95.0%)
        // Notice it calls the inherited calculateAverage() method seamlessly!
        return calculateAverage() >= 95.0;
    }
}
```

- **Parent Method:** We add public boolean qualifiesForHonorRoll() inside the Student class with a base threshold of >= 90.0.
- **Method Overriding:** We copy the exact same method signature inside GraduateStudent, use the **@Override** annotation tag, and change the logic check to >= 95.0.

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — PARENT CLASS BLUEPRINT (Fully Encapsulated) —
class Student {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    private String name;
    private int[] grades;

    // Automated Parent Constructor
    public Student(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    // Public Getters
    public String getName() {
        return this.name;
    }

    public int[] getGrades() {
        return this.grades;
    }

    // Baseline Honor Roll Rule (90.0% or higher)
    public boolean qualifiesForHonorRoll() {
        return calculateAverage() >= 90.0;
    }

    // Private Instance Helpers
    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();

            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("✗ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```

Task 8: OOPS – Polymorphism

Instead of creating separate, rigid code streams for each student type, polymorphism lets you treat both regular Student objects and GraduateStudent objects as if they belong to the same parent type Student.

- Adjust our main method to instantiate a mix of profiles and store them inside a single **Parent Array**. We will then use a single loop to scan the directory, and Java will automatically figure out which honor roll logic rules to apply to each student at runtime.
- We have wrapped the entire application inside an infinite loop (**while (true)**), which allows it to run dynamically for as many students as you want. At the start of each turn, the program asks if you want to add a **Regular Student**, a **Graduate Student**, or **Exit** the system. If you choose to exit, the program safely terminates the loop using the break keyword.

What Changes in This Step:

1. Dynamic Program Loop (while (true)): The application runs continuously until explicitly stopped.
2. User-Driven Object Decisions: An interactive selection menu lets the user choose the specific object type to manufacture on the fly.
3. **Parent Polymorphic Array**: We build a directory array using the parent blueprint type:

```
Student[] schoolRoll = new Student[2];
```
4. **Dynamic Slots**: Slot 0 is assigned a regular Student object, and slot 1 is assigned a GraduateStudent child object. Java allows this because a graduate student *is* a student. [1]
5. **Runtime Magic**: Inside our processing loop, the exact same method call (current.qualifiesForHonorRoll()) switches its underlying engine automatically—applying the 90.0% rule to slot 0 and the stricter 95.0% rule to slot 1 break: Instantly destroys the infinite loop the millisecond choice 3 is entered.
6. continue: Safely throws the application back to the menu start if the user inputs an unexpected integer.
7. instanceof: A new helper keyword used at the end to check: "Is the student currently passing through the engine a graduate student?" If yes, it safely unlocks and displays their hidden thesis title.

This seems like production-grade console application! Congratulation on First project.

Note:

No changes to class Student and class GraduateStudent for adding polymorphism.

Only adjust main method to instantiate a mix of profiles and store them inside a single **Parent Array**.

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println("  WELCOME TO THE GLOBAL STUDENT ENGINE  ");
        System.out.println("=====");

        // — DYNAMIC CONTINUOUS WHILE LOOP —
        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear out leftover newline token from scanner buffer

            // — DYNAMIC EXIT JUMP CONDITION —
            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break; // Instantly breaks the infinite while loop to end the program safely
            }

            // Route user choices or intercept bad choices
            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("✗ Invalid option. Please select 1, 2, or 3.");
                continue; // Skips the rest of this iteration and restarts the menu loop
            }

            // GATHER SHARED BASE PROFILE DETAILS
            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            // Declare a generic parent type pointer variable to handle polymorphism seamlessly
            Student currentStudent = null;

            // — USER DECIDES THE INSTANTIATION TYPE —
            if (menuChoice == 1) {
                // Instantiates a regular student profile
                currentStudent = new Student(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                // Instantiates a specialized graduate student child profile
                currentStudent = new GraduateStudent(rawName, thesis, inputScanner);
            }
        }
    }
}
```

```

// — POLYMORPHIC REPORT COMPILATION —
System.out.println("\n=====");
System.out.println("          GENERATING REPORT CARD          ");
System.out.println("=====");
System.out.println("Account File:    " + currentStudent.getName());
System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

// If the object is a GraduateStudent, show their thesis topic using a safe type cast check
if (currentStudent instanceof GraduateStudent) {
    GraduateStudent castedGrad = (GraduateStudent) currentStudent;
    System.out.println("Research Focus:  " + castedGrad.getThesisTitle());
}

double averageGrade = currentStudent.calculateAverage();
System.out.println("Final Grade:    " + averageGrade + "%");

// Runtime Magic: Automatically triggers the correct 90% or 95% target rules
if (currentStudent.qualifiesForHonorRoll()) {
    System.out.println("Honor Roll:      ★ APPROVED ★");
} else {
    System.out.println("Honor Roll:      Standard Academic Standing");
}
System.out.println("=====");
}

inputScanner.close();
}
}

```

Task 9: File Input Output

Instead of just printing the final report card text to your screen using `System.out`, we will simultaneously write that exact text data to a permanent file named **report_cards.txt**. Each time you add a new student, the program will **append** (add onto) the bottom of that file so your history is never lost.

Note:

No changes to class `Student` and class `GraduateStudent` for adding File Input Output. Only adjust main method to automatically save these report cards to your hard drive, we will introduce **File I/O** (Input/Output) using Java's built-in `FileWriter` and `IOException` error handling.

What We Are Adding in This Final Step:

1. **New Package Imports:** We add `import java.io.FileWriter;` and `import java.io.IOException;` to the very top of our file to pull Java's file-writing tools off the digital library shelf.
2. **The FileWriter Tool:** We open a writer channel inside our report generation segment. By passing `true` as a configuration parameter (`new FileWriter("report_cards.txt", true)`), we tell Java to update and append to the text file rather than wiping it clean every loop cycle.
3. **Mandatory File Try-Catch:** Writing files to a hard drive carries real-world risks (like your hard drive running out of space or being locked). Java forces us to wrap our file-writing code inside a try-catch block to handle an `IOException` safely without crashing.

Verifying Your Work

Once you compile and run this program, add a couple of student records, and select option 3 to exit.

Look inside your project's main folder—you will see a brand-new file named `report_cards.txt` created entirely by your code. If you open it with Notepad or any basic text editor, you will find your clean academic records saved exactly as specified on your disk [local].

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println("  WELCOME TO THE GLOBAL STUDENT ENGINE  ");
        System.out.println("=====");

        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear scanner buffer

            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break;
            }

            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("✗ Invalid option. Please select 1, 2, or 3.");
                continue;
            }

            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            Student currentStudent = null;

            if (menuChoice == 1) {
                currentStudent = new Student(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                currentStudent = new GraduateStudent(rawName, thesis, inputScanner);
            }
        }
    }
}
```

```
// --- POLYMORPHIC REPORT COMPILATION ---
    System.out.println("\n=====");
    System.out.println("          GENERATING REPORT CARD          ");
    System.out.println("=====");
    System.out.println("Account File:    " + currentStudent.getName());
    System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

    String extraDetails = "";
    if (currentStudent instanceof GraduateStudent) {
        GraduateStudent castedGrad = (GraduateStudent) currentStudent;
        extraDetails = "Research Focus:  " + castedGrad.getThesisTitle() + "\n";
        System.out.print(extraDetails);
    }

    double averageGrade = currentStudent.calculateAverage();
    System.out.println("Final Grade:      " + averageGrade + "%");

    String honorStatus = "Standard Academic Standing";
    if (currentStudent.qualifiesForHonorRoll()) {
        honorStatus = "★ APPROVED FOR HONOR ROLL ★";
    }
    System.out.println("Honor Roll:      " + honorStatus);
    System.out.println("=====");

    // — NEW STEP: AUTOMATICALLY SAVE DATA TO HARD DRIVE VIA FILE I/O —
    // Setting the second parameter to 'true' enables APPEND mode so old entries stay saved!
    try (FileWriter fileWriter = new FileWriter("report_cards.txt", true)) {

        // Write data patterns onto the disk exactly how we formatted them on screen
        fileWriter.write("=== STUDENT RECORD ===\n");
        fileWriter.write("Name: " + currentStudent.getName() + "\n");
        fileWriter.write("Average: " + averageGrade + "%\n");
        if (!extraDetails.isEmpty()) {
            fileWriter.write(extraDetails);
        }
        fileWriter.write("Status: " + honorStatus + "\n");
        fileWriter.write("-----\n\n");

        System.out.println("💾 Backup Status: Report successfully written to 'report_cards.txt'.");
    } catch (IOException e) {
        // Intercept data path anomalies (like read-only file lockouts or hardware drops)
        System.out.println("❌ Local Storage Failure: Could not write record backup to disk drive.");
    }
}

inputScanner.close();
}
```

Task 10: Read back from on start-up.

- Read your saved records back into the program when the application boots up, use Java's built-in File and Scanner tools.
- Instead of jumping straight into the interactive loop, the program will look for report_cards.txt the exact moment it starts. If the file exists, the program should open it, read it line-by-line, and print the historical records directly to the screen before presenting the registration menu.

What We Are Adding in This Step:

1. **New Package Import:** We add import java.io.File; to manage file path lookups on your operating system.
2. **Boot-Up Initializer Function:** We introduce a new public static method called **loadPastRecords()** outside of main.
3. **The File Reading Process:** We pass a File object into a new Scanner. The scanner uses a while (fileScanner.hasNextLine()) loop to read every single line of text from the file until it hits the very end.
4. **Zero Code Discarded:** The FileWriter append logic, object structure, and error-handling mechanics remain completely active.

How to Test this Evolution:

1. Compile and run the code. If it's your very first time running it with this updated logic, it will say "No historical database found"
2. Add a student (e.g., "Sam Wilson") and let the program log their grades to disk.
3. Exit the application using choice 3.
4. Run the program a **second time**. Notice what happens instantly on boot-up! The loadPastRecords() block finds the file and outputs Sam's entire profile right back onto your console interface before asking you for new inputs.

Future improvements

- Learn about ArrayLists: Upgrade UnifiedTracker.java so that instead of just flashing reports on screen, the system keeps an active in-memory list tracking every student registered during that loop cycle.
- Log error for duplicate entry.
- Add student member variables for student ID, Second name etc.
- Add UI for user interaction.

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {

    // — NEW STEP: FUNCTION TO LOAD PAST RECORDS FROM LOCAL DISK —
    public static void loadPastRecords() {
        File databaseFile = new File("report_cards.txt");

        // 1. Check if the backup file actually exists on the computer
        if (!databaseFile.exists()) {
            System.out.println("❗ No historical database found. Starting with a blank registry.\n");
            return; // Exit function early if there is nothing to read
        }

        System.out.println("📁 Loading historical archives from disk drive...");
        System.out.println("=====");

        // 2. Open a specialized scanner locked onto the physical file instead of System.in
        try (Scanner fileScanner = new Scanner(databaseFile)) {

            // 3. Keep loop active as long as there are text strings left to read in the file
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line); // Output historical file lines directly to console
            }
            System.out.println("=====");
            System.out.println("✅ Data successfully loaded into memory view!\n");

        } catch (IOException e) {
            // Guard against system errors during active read runtime
            System.out.println("❌ Local Storage Failure: Could not safely parse text files.");
        }
    }

    public static void main(String[] args) {
        // — BOOT-UP REUSE: FETCH OLD HISTORY THE MOMENT APPLICATION STARTS —
        loadPastRecords();
    }
}
```